



S8-Trabajo Autónomo 2P

[RESUMEN DEL CURSO](#)[S1 - Semana Informativa](#)[S2 - Semana 1-2](#)[S3 - Semana 3-4](#)[S4 - Semana 5-6](#)[S5 - Semana 7-8](#)[S6 - Semana 9-10](#)[S7 - Semana 11-12](#)[S8 - Semana 13-14](#)[S9 - Semana 15-16](#)[S10 - Evaluación final](#)[S11 - Mejoramiento](#)[CLASES GRABADAS](#)

Por hacer: Hacer un envío

Apertura: lunes, 20 de julio de 2026, 00:00

TAU2

DESARROLLO DE UNA APLICACIÓN DOMÓTICA CON ARDUINO, NODE.JS Y REACT

Asignatura

Reutilización de Software

Modalidad

Trabajo grupal.

Tema

Desarrollo de una aplicación domótica para el monitoreo ambiental, control de iluminación, medición de variables eléctricas y exportación de datos a Excel.

1. Descripción de la actividad

Los estudiantes deberán diseñar, desarrollar e implementar una aplicación web de estilo domótico que permita adquirir, almacenar, visualizar y exportar datos obtenidos mediante Arduino.

La solución deberá integrar los siguientes componentes:

- Arduino para la adquisición o simulación de datos.
- Backend desarrollado con Node.js.
- API REST para la comunicación entre los diferentes componentes.
- Base de datos para almacenar las mediciones.
- Frontend desarrollado con React.
- Dashboard para la visualización de la información.
- Módulo para exportar el historial de datos a un archivo Excel.

La aplicación deberá demostrar el uso de componentes, servicios, controladores, modelos, validaciones y funciones reutilizables.

2. Objetivo general

Desarrollar una aplicación domótica modular que integre Arduino, Node.js, React y una base de datos, aplicando principios de reutilización de software para la adquisición, almacenamiento, visualización, control y exportación de datos.

3. Objetivos específicos

- Adquirir datos de temperatura y humedad ambiental mediante Arduino.
- Obtener o simular datos de voltaje y amperaje de un dispositivo.

- Transmitir los datos desde Arduino hacia el backend.
- Desarrollar una API REST utilizando Node.js.
- Utilizar Knex.js para la gestión de migraciones y conexión con la base de datos.
- Utilizar Objection.js para la creación de modelos y consultas.
- Validar los datos recibidos mediante Joi.
- Utilizar Nodemon durante el desarrollo del backend.
- Almacenar las mediciones en una base de datos.
- Desarrollar un dashboard utilizando React.
- Implementar componentes reutilizables en el frontend.
- Simular el encendido y apagado de un foco desde la interfaz.
- Exportar el historial de mediciones a un archivo Excel.
- Aplicar una arquitectura modular y organizada.

4. Tecnologías obligatorias

4.1 Backend

El backend deberá desarrollarse utilizando obligatoriamente:

- Node.js.
- Express.
- Knex.js.
- Objection.js.
- Joi.
- Nodemon.
- Dotenv.
- CORS.
- Librería para comunicación serial o HTTP con Arduino.
- Librería para generación de archivos Excel.

Para la exportación se podrá utilizar una librería como:

- ExcelJS.
- SheetJS.
- XLSX.

4.2 Frontend

El frontend deberá desarrollarse utilizando:

- React.
- Componentes funcionales.
- React Hooks.
- Axios o Fetch API.
- Una librería para gráficos, como Chart.js, Recharts o ApexCharts.
- CSS, Bootstrap, Material UI u otra librería de estilos.

4.3 Base de datos

Se podrá utilizar una de las siguientes bases de datos:

- MySQL.
- MariaDB.
- PostgreSQL.
- SQLite.

La base de datos deberá ser gestionada desde el backend mediante Knex.js y Objection.js.

4.4 Hardware

Se podrá utilizar:

- Arduino Uno.
- Arduino Nano.
- Arduino Mega.
- ESP32.
- ESP8266.
- Otra placa compatible, previa autorización del docente.

Elementos sugeridos:

- Sensor DHT11 o DHT22.
- Sensor de voltaje.
- Sensor de corriente.
- LED para representar el foco.
- Resistencia de 220 Ω .
- Protoboard.
- Cables de conexión.

Cuando no se disponga de todos los sensores, los valores podrán ser simulados desde Arduino. No se permitirá generar los datos directamente desde React.

5. Arquitectura general de la aplicación

La aplicación deberá implementar el siguiente flujo:

```
Arduino y sensores
  |
  | Comunicación serial o HTTP
  v
Backend Node.js
  |
  | Validación con Joi
  |
  | Knex.js y Objection.js
  v
Base de datos
  |
  | API REST
  v
Frontend React
  |
  v
Dashboard domótico
  |
  v
Exportación de datos a Excel
```

6. Datos que deberá adquirir el sistema

Arduino deberá adquirir o simular los siguientes datos:

- Temperatura ambiental en grados Celsius.
- Humedad ambiental en porcentaje.
- Voltaje de un dispositivo en voltios.
- Amperaje o corriente de un dispositivo en amperios.
- Estado del foco: encendido o apagado.

Ejemplo de datos enviados por Arduino:

```
{
  "temperatura": 24.5,
  "humedad": 65,
  "voltaje": 12.1,
  "amperaje": 1.8,
  "estadoFoco": true
}
```

El backend podrá calcular adicionalmente la potencia eléctrica mediante la siguiente fórmula:

Potencia = Voltaje × Amperaje

La potencia deberá expresarse en vatios.

7. Requerimientos del backend

El backend deberá:

- Recibir los datos enviados por Arduino.
- Procesar los datos en formato JSON.
- Validar la información mediante Joi.
- Rechazar valores incorrectos o fuera de rango.
- Calcular la potencia eléctrica.
- Almacenar las mediciones en la base de datos.
- Consultar la medición más reciente.
- Consultar el historial de mediciones.
- Controlar el estado del foco.
- Enviar órdenes de encendido o apagado hacia Arduino.
- Generar un archivo Excel con el historial.
- Gestionar los errores de forma centralizada.
- Utilizar variables de entorno.
- Organizar el código mediante rutas, controladores, servicios, modelos y validaciones.

8. Requerimientos del frontend

El frontend deberá mostrar un dashboard que contenga como mínimo:

- Temperatura ambiental actual.
- Humedad ambiental actual.
- Voltaje actual.
- Amperaje actual.
- Potencia eléctrica calculada.
- Estado actual del foco.
- Botón para encender el foco.
- Botón para apagar el foco.
- Fecha y hora de la última medición.
- Indicador de conexión con el backend.
- Gráfico histórico de temperatura.
- Gráfico histórico de humedad.
- Gráfico histórico de voltaje.
- Gráfico histórico de amperaje.
- Botón para exportar los datos a Excel.

El diseño deberá ser ordenado, comprensible y adaptable a diferentes tamaños de pantalla.

9. Exportación de datos a Excel

La aplicación deberá permitir descargar el historial de mediciones en un archivo con extensión:

.xlsx

El archivo deberá generarse desde el backend y descargarse desde un botón ubicado en el frontend.

El archivo Excel deberá incluir como mínimo las siguientes columnas:

Columna	Descripción
ID	Identificador de la medición
Fecha	Fecha de registro
Hora	Hora de registro
Temperatura	Temperatura en grados Celsius
Humedad	Humedad en porcentaje
Voltaje	Voltaje en voltios
Amperaje	Corriente en amperios
Potencia	Potencia en vatios
Estado del foco	Encendido o apagado

El archivo deberá cumplir con las siguientes condiciones:

- Tener un nombre descriptivo.
- Incluir encabezados.
- Contener los datos ordenados.
- Mostrar correctamente las unidades.
- Descargar datos reales almacenados en la base de datos.
- No utilizar datos escritos manualmente.
- Generarse mediante una ruta del backend.

Ejemplo del nombre del archivo:

reporte_domotico_2026-07-18.xlsx

Endpoint sugerido:

GET /api/measurements/export/excel

Al seleccionar el botón **Exportar a Excel**, el navegador deberá descargar automáticamente el archivo.

10. Estructura sugerida del backend

```
backend/
├── src/
│   ├── config/
│   │   ├── database.js
│   │   └── environment.js
│   ├── controllers/
│   │   ├── measurementController.js
│   │   ├── deviceController.js
│   │   └── exportController.js
│   ├── models/
│   │   ├── Measurement.js
│   │   └── Device.js
│   ├── routes/
│   │   ├── measurementRoutes.js
│   │   ├── deviceRoutes.js
│   │   └── exportRoutes.js
│   └── services/
```

```
| | | | measurementService.js
| | | | deviceService.js
| | | | excelService.js
| | | |
| | | | validations/
| | | | | measurementSchema.js
| | | | | deviceSchema.js
| | | | |
| | | | | middlewares/
| | | | | | validateRequest.js
| | | | | | errorHandler.js
| | | | | | notFound.js
| | | | |
| | | | | serial/
| | | | | | arduinoConnection.js
| | | | |
| | | | | app.js
| | | | | server.js
| | | |
| | | | migrations/
| | | |
| | | | seeds/
| | | |
| | | | knexfile.js
| | | |
| | | | .env
| | | |
| | | | .env.example
| | | |
| | | | package.json
```

11. Estructura sugerida del frontend

```
frontend/
| | src/
| | | components/
| | | | MeasurementCard.jsx
| | | | StatusIndicator.jsx
| | | | DeviceControl.jsx
| | | | SensorChart.jsx
| | | | ExportButton.jsx
| | | | ConnectionStatus.jsx
| | |
| | | hooks/
| | | | useSensorData.js
| | |
| | | services/
| | | | domoticApi.js
| | |
| | | pages/
| | | | Dashboard.jsx
| | |
| | | App.jsx
| | |
| | | main.jsx
| |
| | package.json
```

12. Modelo de datos sugerido

12.1 Tabla de mediciones

Nombre sugerido:

measurements		
Campo	Tipo	Descripción
id	Entero	Identificador de la medición
temperature	Decimal	Temperatura ambiental
humidity	Decimal	Humedad ambiental
voltage	Decimal	Voltaje medido
current	Decimal	Amperaje medido
power	Decimal	Potencia calculada
created_at	Fecha y hora	Fecha de registro
updated_at	Fecha y hora	Fecha de actualización

12.2 Tabla de dispositivos

Nombre sugerido:

devices		
Campo	Tipo	Descripción
id	Entero	Identificador del dispositivo
name	Texto	Nombre del dispositivo
type	Texto	Tipo de dispositivo
status	Booleano	Estado encendido o apagado
created_at	Fecha y hora	Fecha de creación
updated_at	Fecha y hora	Fecha de actualización

13. Endpoints mínimos de la API

13.1 Registrar una medición

POST /api/measurements

Ejemplo de cuerpo:

```
{
  "temperature": 24.5,
  "humidity": 65,
  "voltage": 12.1,
  "current": 1.8
}
```

13.2 Consultar la última medición

GET /api/measurements/latest

13.3 Consultar el historial

GET /api/measurements

Consulta con límite:

GET /api/measurements?limit=50

13.4 Consultar el estado del foco

GET /api/devices/light

13.5 Cambiar el estado del foco

PATCH /api/devices/light

Para encender:

```
{
  "status": true
}
```

Para apagar:

```
{
  "status": false
}
```

13.6 Exportar datos a Excel

GET /api/measurements/export/excel

14. Validaciones con Joi

El backend deberá validar como mínimo:

- Que la temperatura sea numérica.
- Que la humedad esté entre 0 y 100.
- Que el voltaje sea igual o mayor que cero.
- Que el amperaje sea igual o mayor que cero.
- Que el estado del foco sea booleano.
- Que no existan campos obligatorios vacíos.
- Que los mensajes de error sean comprensibles.

Ejemplo:

```
const Joi = require("joi");

const measurementSchema = Joi.object({
  temperature: Joi.number().required(),
  humidity: Joi.number().min(0).max(100).required(),
  voltage: Joi.number().min(0).required(),
  current: Joi.number().min(0).required()
});

module.exports = measurementSchema;
```

15. Comunicación con Arduino

La comunicación entre Arduino y Node.js podrá realizarse mediante una de las siguientes alternativas.

Alternativa A: comunicación serial

Arduino enviará los datos mediante el puerto USB.

Ejemplo:

```
{"temperature":24.5,"humidity":65,"voltage":12.1,"current":1.8}
```

Node.js deberá leer y procesar la información mediante una librería de comunicación serial.

Alternativa B: comunicación HTTP

Cuando se utilice ESP32 o ESP8266, la placa podrá enviar los datos directamente hacia la API REST mediante solicitudes HTTP.

Alternativa C: simulación desde Arduino

Cuando no se disponga de sensores físicos, Arduino podrá generar valores simulados.

Los datos simulados deberán originarse en el código de Arduino y ser enviados al backend.

No se permitirá generar las mediciones directamente desde React.

16. Control del foco

El foco podrá representarse mediante un LED conectado a una salida digital.

El funcionamiento esperado será:

1. El usuario selecciona el botón de encendido o apagado en React.
2. React envía una solicitud al backend.
3. El backend valida la solicitud.
4. El backend registra el nuevo estado.
5. Node.js envía la orden correspondiente a Arduino.
6. Arduino enciende o apaga el LED.
7. El backend devuelve el estado actualizado.
8. React actualiza el indicador visual.

Comandos sugeridos:

```
LIGHT_ON  
LIGHT_OFF
```

17. Aplicación de reutilización de software

El proyecto deberá demostrar claramente la reutilización de componentes y módulos.

17.1 Reutilización en el backend

Se deberán implementar elementos como:

- Middleware reutilizable para validar datos con Joi.
- Middleware centralizado para manejo de errores.
- Servicio reutilizable para acceso a la base de datos.
- Modelo base mediante Objection.js.
- Configuración centralizada de Knex.
- Servicio independiente para generación de Excel.
- Funciones comunes para respuestas HTTP.
- Variables de entorno reutilizables.
- Separación entre rutas, controladores, servicios y modelos.

17.2 Reutilización en el frontend

Se deberán implementar elementos como:

- Componente reutilizable para tarjetas de medición.
- Componente reutilizable para gráficos.
- Componente reutilizable para botones.
- Componente reutilizable para mostrar estados.
- Componente reutilizable para exportar datos.
- Servicio centralizado para consumir la API.
- Hook personalizado para obtener las mediciones.
- Componente reutilizable para mostrar errores y estados de carga.

Cada grupo deberá explicar durante la exposición:

- Qué componentes fueron reutilizados.
- En qué partes se utilizaron.
- Qué duplicación de código evitaron.
- Cómo mejoran el mantenimiento.
- Cómo podrían ser utilizados en otra aplicación.

18. Requisitos funcionales

- El sistema deberá adquirir datos desde Arduino.
- El sistema deberá registrar temperatura y humedad.
- El sistema deberá registrar voltaje y amperaje.
- El sistema deberá calcular la potencia.
- El backend deberá almacenar las mediciones.
- El frontend deberá mostrar los datos actuales.
- El frontend deberá mostrar gráficos históricos.
- El usuario deberá poder encender y apagar un foco.
- El estado del foco deberá sincronizarse entre Arduino, backend y frontend.
- El backend deberá validar la información mediante Joi.
- El sistema deberá exportar el historial a Excel.
- El frontend deberá informar errores de conexión.
- El sistema deberá mostrar la fecha y hora de la última medición.

19. Requisitos no funcionales

- La aplicación deberá utilizar una arquitectura modular.
- El código deberá estar organizado y documentado.
- Los nombres de variables y funciones deberán ser descriptivos.
- Las credenciales deberán almacenarse en variables de entorno.
- El dashboard deberá ser responsivo.
- La API deberá responder en formato JSON.
- La base de datos deberá utilizar migraciones de Knex.
- Los modelos deberán implementarse con Object.js.
- Los datos deberán validarse mediante Joi.
- El backend deberá ejecutarse durante el desarrollo con Nodemon.
- Los errores deberán manejarse de manera centralizada.
- El archivo Excel deberá generarse desde el backend.
- La interfaz deberá mostrar mensajes comprensibles al usuario.

20. Pruebas mínimas obligatorias

Los estudiantes deberán comprobar como mínimo los siguientes casos:

Prueba	Resultado esperado
Envío de datos válidos	La medición se almacena correctamente
Humedad mayor a 100	Joi rechaza la solicitud
Campo obligatorio vacío	La API devuelve un mensaje de error
Consulta de última medición	Se obtiene el registro más reciente
Consulta del historial	Se obtiene una lista de mediciones
Encendido del foco	Arduino activa el LED
Apagado del foco	Arduino desactiva el LED
Desconexión de Arduino	Se informa la pérdida de conexión
Desconexión del backend	React muestra un mensaje de error
Exportación a Excel	Se descarga un archivo válido
Verificación del Excel	El archivo contiene las mediciones registradas
Reinicio con Nodemon	El servidor se reinicia al modificar el código

21. Organización de los grupos

La actividad podrá desarrollarse en grupos de tres o cuatro integrantes.

Roles sugeridos:

- Responsable de Arduino y sensores.
- Responsable del backend.
- Responsable del frontend.
- Responsable de base de datos, pruebas y documentación.

La asignación de roles no limita la responsabilidad individual.

Todos los integrantes deberán conocer el funcionamiento completo de la aplicación.

22. Entregables

Los únicos entregables de la actividad serán:

22.1 Informe técnico

Cada grupo deberá presentar un informe que contenga:

1. Portada.
2. Introducción.
3. Descripción del problema.
4. Objetivo general.
5. Objetivos específicos.
6. Requisitos funcionales.
7. Requisitos no funcionales.
8. Arquitectura de la aplicación.
9. Diagrama de componentes.
10. Diagrama del flujo de datos.
11. Modelo de la base de datos.
12. Tecnologías utilizadas.
13. Descripción del circuito de Arduino.
14. Desarrollo del backend.
15. Uso de Knex.js.
16. Uso de Objection.js.
17. Uso de Joi.
18. Uso de Nodemon.
19. Desarrollo del frontend en React.
20. Componentes reutilizables implementados.
21. Proceso de control del foco.
22. Proceso de exportación a Excel.
23. Pruebas realizadas.
24. Evidencias de funcionamiento.
25. Problemas encontrados y soluciones aplicadas.
26. Conclusiones.
27. Recomendaciones.
28. Bibliografía en formato APA séptima edición.

El informe deberá incluir capturas legibles del sistema funcionando.

22.2 Exposición y demostración práctica

Cada grupo deberá realizar una exposición en la que demuestre:

- Funcionamiento del circuito de Arduino.
- Lectura o simulación de temperatura.
- Lectura o simulación de humedad.
- Lectura o simulación de voltaje.
- Lectura o simulación de amperaje.
- Recepción de datos en Node.js.
- Validación mediante Joi.
- Registro de datos en la base de datos.
- Consultas mediante Objection.js.
- Migraciones realizadas con Knex.js.
- Dashboard desarrollado en React.
- Visualización de los gráficos.
- Encendido y apagado del foco.
- Actualización del estado en el dashboard.
- Exportación de los datos a Excel.
- Apertura y verificación del archivo Excel.
- Componentes y módulos reutilizados.

La exposición deberá ser realizada por todos los integrantes del grupo.

La participación y defensa serán evaluadas individualmente.

23. Restricciones

- No se permitirá desarrollar todo el backend en un único archivo.
 - No se permitirá realizar consultas SQL directamente en los controladores.
 - No se permitirá omitir Knex.js.
 - No se permitirá omitir Objection.js.
 - No se permitirá omitir Joi.
 - No se permitirá omitir Nodemon.
 - No se permitirá almacenar credenciales directamente en el código.
 - No se permitirá generar los datos únicamente desde React.
 - Los datos deberán originarse en Arduino.
 - No se permitirá duplicar componentes que puedan reutilizarse.
 - No se permitirá presentar únicamente capturas sin demostrar la aplicación.
 - No se permitirá exportar un archivo Excel con datos escritos manualmente.
 - No se permitirá que un solo integrante realice toda la exposición.
 - No se permitirá presentar una aplicación que no almacene información en una base de datos.
-

24. Rúbrica de evaluación

La actividad será evaluada sobre 10 puntos.

Criterio	Puntaje
Adquisición o simulación de datos desde Arduino	1,00
Backend desarrollado con Node.js y arquitectura modular	1,00
Uso correcto de Knex.js y Objection.js	1,00
Validación de datos mediante Joi	0,50
Base de datos, migraciones y almacenamiento	0,75
Dashboard desarrollado en React	1,00
Visualización de temperatura, humedad, voltaje y amperaje	0,75
Control de encendido y apagado del foco	0,75
Exportación correcta de los datos a Excel	0,75
Reutilización de componentes y módulos	0,75
Informe técnico	0,75
Exposición y defensa individual	1,00
Total	10,00

25. Criterios para la defensa

Durante la exposición, el docente podrá solicitar a cualquier integrante:

- Explicar el flujo de datos desde Arduino hasta React.
- Mostrar cómo Arduino envía los datos.
- Modificar una validación de Joi.
- Explicar una migración de Knex.
- Realizar una consulta con Objection.js.
- Explicar un controlador o servicio.
- Agregar un campo al modelo.
- Modificar una tarjeta del dashboard.
- Encender o apagar el foco.
- Explicar el cálculo de potencia.
- Mostrar la ruta que genera el archivo Excel.
- Modificar una columna del archivo Excel.
- Explicar un componente reutilizable.
- Solucionar una falla de comunicación.
- Mostrar el contenido almacenado en la base de datos.

La calificación de la defensa será individual.

26. Resultado esperado

Al finalizar la actividad, el sistema deberá permitir que el usuario acceda a un dashboard y visualice:

- Temperatura ambiental.
- Humedad ambiental.
- Voltaje.
- Amperaje.
- Potencia eléctrica.
- Estado del foco.
- Historial de mediciones.
- Gráficos de las variables.
- Fecha y hora de la última actualización.
- Estado de conexión.
- Botones para encender y apagar el foco.
- Botón para exportar los datos a Excel.

La aplicación deberá demostrar que los datos se originan en Arduino, son procesados por Node.js, validados con Joi, almacenados mediante Knex.js y Objection.js, visualizados en React y exportados correctamente a un archivo Excel.

Agregar entrega

Estado de la entrega

Número del intento	Este es el intento 1.
--------------------	-----------------------

Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar
Última modificación	-
Comentarios de la entrega	> Comentarios (0) .